

AVL Tree Implementation

Complete Line-by-Line Code Explanation

Viva Preparation Guide

Data Structures | C++ | `avl.h` `avl.cpp` `main.cpp`

1. Project Overview

This project implements a self-balancing AVL Tree in C++ across three files: avl.h (header / class declarations), avl.cpp (all method definitions), and main.cpp (the interactive menu-driven program). The tree automatically re-balances itself after every insert or delete using rotations, keeping all operations at $O(\log n)$ time complexity.

The implementation covers 10 features beyond basic BST:

- **Feature 1:** Duplicate-aware insert & delete (count field)
- **Feature 2:** Non-recursive traversals using explicit stacks
- **Feature 3:** Reverse AVL Tree (descending order)
- **Feature 4:** Student Record storage (Roll No, Name, Marks)
- **Feature 5:** Leaf node display and bulk deletion
- **Feature 6:** Inorder Successor & Predecessor
- **Feature 7:** Print all nodes at a given level
- **Feature 8:** Display all left-child nodes
- **Feature 9:** Find largest value at a given level
- **Feature 10:** Display all right-child nodes

2. avl.h — Header File Explained

2.1 Include Guards

```
#ifndef AVL_H
#define AVL_H
...
#endif
```

These three lines form an include guard. If this header file is included in more than one .cpp file, the compiler would see the declarations twice and throw a 'redefinition' error. `#ifndef` checks whether `AVL_H` is not yet defined; if true, it defines it and proceeds. `#endif` closes the guard. Any subsequent inclusion of the same file is silently skipped.

2.2 Includes and Namespace

```
#include <iostream>    // cin, cout
#include <stack>        // std::stack used in non-recursive traversals
#include <string>        // std::string for student names
using namespace std; // avoids writing std:: everywhere
```

2.3 struct node — The Tree Node

```
struct node {
    int    info;    // The stored key (or Roll No in student mode)
    int    count;   // Counts duplicates; starts at 1
    string name;    // Student name — empty string in integer mode
    float  marks;   // Student marks — 0.0 in integer mode
    int    height; // AVL height of this node; leaf = 1
    struct node *lchild; // Pointer to left child
    struct node *rchild; // Pointer to right child
};
```

A struct node is the fundamental building block of the tree. Every node in memory looks like this. The height field is extra compared to a plain BST — AVL needs it to detect imbalance. count makes the same node reusable for duplicate values instead of inserting another node.

2.4 struct RevNode — Reverse Tree Node

```
struct RevNode {
    int    info;
    RevNode *lchild;
    RevNode *rchild;
};
```

A lighter node used exclusively by ReverseAVLTree. It has no height, count, name, or marks because the reverse tree is only used to print values in descending order — no AVL balancing, no student data needed.

2.5 class ReverseAVLTree

```
class ReverseAVLTree {
public:
    RevNode *root;
    ReverseAVLTree();
    void insert(int item);
    void inorder(RcvNode *ptr); // prints descending
};
```

This class stores a separate tree where the BST rule is inverted: larger values go to the LEFT child instead of the right. As a result, a standard inorder traversal (Left -> Root -> Right) visits nodes from largest to smallest, giving a descending output.

2.6 class AVLTree — Main Class

The main AVLTree class has a public root pointer and two groups of methods:

Public interface methods (called from main.cpp)

- search, insert, deletion — core BST + AVL operations
- case_a / case_b / case_c — kept for backward compatibility; deprecated in AVL mode
- preorder / inorder / postorder — recursive traversals
- insertDup / deletionDup — Feature 1: duplicate handling
- inorderNR / preorderNR / postorderNR — Feature 2: non-recursive
- insertStudent / searchStudent — Feature 4: student records
- displayLeaves / deleteLeaves — Feature 5: leaf operations
- findSuccessorPredecessor — Feature 6
- nodesAtLevel — Feature 7
- displayLeftNodes — Feature 8
- largestAtLevel — Feature 9
- displayRightNodes — Feature 10

Private helper methods

- getHeight — safely reads height (returns 0 for NULL)
- getBalanceFactor — left height minus right height
- leftRotate / rightRotate — the two AVL rotation primitives
- insertAVL / deleteAVL — recursive helpers that do the real balancing work
- findMin — walks left to find the smallest node in a subtree
- deleteLeavesHelper — recursive helper for bulk leaf deletion

3. avl.cpp — Core AVL Mechanics

3.1 Constructor

```
AVLTree::AVLTree() {  
    root = NULL;  
}
```

Sets the root pointer to NULL so the tree starts empty. This is called automatically when you declare `AVLTree avlTree;` in `main()`.

3.2 getHeight()

```
int AVLTree::getHeight(node *ptr) {  
    if (ptr == NULL) return 0;  
    return ptr->height;  
}
```

A safe wrapper around `ptr->height`. Without this check, dereferencing a NULL pointer would crash the program. Returning 0 for NULL means a missing child contributes 0 to height calculations, which is mathematically correct.

3.3 getBalanceFactor()

```
int AVLTree::getBalanceFactor(node *ptr) {  
    if (ptr == NULL) return 0;  
    return getHeight(ptr->lchild) - getHeight(ptr->rchild);  
}
```

Balance Factor (BF) = height of left subtree minus height of right subtree. AVL property requires BF to be -1, 0, or +1 for every node. If BF becomes +2 the left side is too tall; if BF becomes -2 the right side is too tall. Either case triggers a rotation.

3.4 leftRotate() — Right-Heavy Imbalance Fix

```
node* AVLTree::leftRotate(node *ptr) {  
    node *temp = ptr->rchild;    // Step 1: temp = right child  
    ptr->rchild = temp->lchild;    // Step 2: move temp's left subtree to ptr's  
    right  
    temp->lchild = ptr;           // Step 3: ptr becomes left child of temp  
    // Step 4: update heights bottom-up  
    ptr->height = 1 + max(getHeight(ptr->lchild), getHeight(ptr->rchild));  
    temp->height = 1 + max(getHeight(temp->lchild), getHeight(temp->rchild));  
    return temp; // temp is the new root of this subtree  
}
```

Imagine the tree is leaning right. `leftRotate` lifts the right child (`temp`) up to become the new parent, and the original parent (`ptr`) slides down to become the left child of `temp`. Heights are always recalculated bottom-up because `ptr`'s height changes first, and then `temp` depends on it.

3.5 rightRotate() — Left-Heavy Imbalance Fix

```
node* AVLTree::rightRotate(node *ptr) {
    node *temp = ptr->lchild;    // temp = left child
    ptr->lchild = temp->rchild;    // move temp's right subtree to ptr's left
    temp->rchild = ptr;          // ptr becomes right child of temp
    ptr->height = 1 + max(getHeight(ptr->lchild), getHeight(ptr->rchild));
    temp->height = 1 + max(getHeight(temp->lchild), getHeight(temp->rchild));
    return temp;
}
```

Mirror image of leftRotate. Used when the tree is leaning left (balance factor > 1). The left child becomes the new parent, and the original node slides down to be its right child.

3.6 findMin()

```
node* AVLTree::findMin(node *ptr) {
    if (ptr == NULL) return NULL;
    while (ptr->lchild != NULL) ptr = ptr->lchild;
    return ptr;
}
```

In a BST, the minimum value is always at the leftmost node. We keep going left until there is no more left child. Used during deletion: when deleting a node with two children, we replace it with its inorder successor, which is the minimum of its right subtree.

3.7 search()

```
node* AVLTree::search(int item, node *ptr) {
    if (ptr == NULL) return NULL;    // not found
    if (item == ptr->info) return ptr;    // found
    else if (item < ptr->info)
        return search(item, ptr->lchild);    // go left
    else
        return search(item, ptr->rchild);    // go right
}
```

Classic recursive BST search. At each node, compare item with ptr->info. If equal, return this node. If item is smaller, the answer must be in the left subtree (BST property: left < root). If item is larger, go right. Returns NULL if the item is not found, which many other functions use as a 'not found' signal.

4. insertAVL() — The Heart of AVL Insertion

insertAVL is a private recursive function. The public insert() simply calls `root = insertAVL(root, item)`. This pattern of returning the new subtree root is important — it lets each recursive call update its parent's child pointer cleanly.

4.1 Base Case — Create a New Leaf

```
if (ptr == NULL) {
    node *tmp = new node();
    tmp->info  = item;
    tmp->count = 1;
    tmp->name  = "";
    tmp->marks = 0.0f;
    tmp->height = 1;          // a single leaf has height 1
    tmp->lchild = tmp->rchild = NULL;
    return tmp;
}
```

When we reach a NULL slot, we create the new node here. `height = 1` because a single node with no children has height 1 by convention. `count` starts at 1 meaning one copy of this value. `name` and `marks` are blank because this is integer mode.

4.2 Recursive Descent

```
if (item < ptr->info)
    ptr->lchild = insertAVL(ptr->lchild, item);
else if (item > ptr->info)
    ptr->rchild = insertAVL(ptr->rchild, item);
else { cout << "Item already present"; return ptr; }
```

Navigate down exactly like a regular BST. Smaller values go left, larger go right. If equal, it is a duplicate and we refuse the insert (duplicate handling is done separately by `insertDup`). By assigning the return value back to `ptr->lchild` or `ptr->rchild`, we capture any rotation that might happen deeper in the tree.

4.3 Height Update

```
ptr->height = 1 + max(getHeight(ptr->lchild), getHeight(ptr->rchild));
```

After the recursive call returns, this node's height may have increased because a new leaf was added somewhere below it. We recalculate: `my height = 1` (for myself) + the taller of my two children. This propagates upward as the call stack unwinds.

4.4 Four AVL Imbalance Cases

After updating height, we check the balance factor and handle all four possible imbalances:

Case 1: Left-Left (LL) — $BF > 1$ and item inserted in left subtree of left child

```
if (balance > 1 && item < ptr->lchild->info)
    return rightRotate(ptr);
```

The tree is left-heavy AND the new node is in the left subtree of the left child. One rightRotate on the current node fixes it.

Case 2: Right-Right (RR) — $BF < -1$ and item inserted in right subtree of right child

```
if (balance < -1 && item > ptr->rchild->info)
    return leftRotate(ptr);
```

Mirror of LL. The tree is right-heavy AND the new node is in the right subtree of the right child. One leftRotate fixes it.

Case 3: Left-Right (LR) — $BF > 1$ and item in right subtree of left child

```
if (balance > 1 && item > ptr->lchild->info) {
    ptr->lchild = leftRotate(ptr->lchild);
    return rightRotate(ptr);
}
```

Requires two rotations. First leftRotate on the left child converts it to an LL case, then rightRotate on the current node finishes the fix.

Case 4: Right-Left (RL) — $BF < -1$ and item in left subtree of right child

```
if (balance < -1 && item < ptr->rchild->info) {
    ptr->rchild = rightRotate(ptr->rchild);
    return leftRotate(ptr);
}
```

Mirror of LR. rightRotate on the right child converts to RR, then leftRotate on current node fixes it.

5. deleteAVL() — AVL Deletion with Rebalancing

deleteAVL is also private and recursive. The public deletion() first checks if the item exists (using search), then calls `root = deleteAVL(root, item)`.

5.1 Navigate to the Node

```
if (item < ptr->info)
    ptr->lchild = deleteAVL(ptr->lchild, item);
else if (item > ptr->info)
    ptr->rchild = deleteAVL(ptr->rchild, item);
```

Same recursive navigation as insert. Go left or right until we find the target.

5.2 Three Cases When Node is Found

Case A — Leaf node (no children)

```
if (ptr->lchild == NULL && ptr->rchild == NULL) {
    delete ptr;
    return NULL;
}
```

Simply free the memory and return NULL to the parent so it knows its child pointer should be set to NULL.

Case B — One child

```
node *temp = (ptr->lchild != NULL) ? ptr->lchild : ptr->rchild;
delete ptr;
return temp;
```

The ternary operator picks whichever child exists. We delete the current node and return the surviving child to replace it. The parent then points directly to the grandchild.

Case C — Two children (the complex case)

```
node *temp = findMin(ptr->rchild); // find inorder successor
ptr->info = temp->info;             // copy data from successor
ptr->count = temp->count;
ptr->name = temp->name;
ptr->marks = temp->marks;
ptr->rchild = deleteAVL(ptr->rchild, temp->info); // delete successor
```

We do NOT actually delete the current node. Instead, we find its inorder successor (smallest value in the right subtree), copy its data into the current node, and then recursively delete the successor from the right subtree. The inorder successor always has at most one child (it has no left child by definition), so deleting it falls into Case A or B — simpler situations.

5.3 Rebalance After Deletion

After deletion, the four rotation cases are checked again, but the conditions use `getBalanceFactor` of the child rather than comparing the deleted item's position:

```
// LL: left heavy and left child is not right-heavy
if (balance > 1 && getBalanceFactor(ptr->lchild) >= 0)
    return rightRotate(ptr);
```

```
// LR: left heavy and left child is right-heavy
if (balance > 1 && getBalanceFactor(ptr->lchild) < 0) {
    ptr->lchild = leftRotate(ptr->lchild);
    return rightRotate(ptr);
}
```

```
// RR: right heavy and right child is not left-heavy
if (balance < -1 && getBalanceFactor(ptr->rchild) <= 0)
    return leftRotate(ptr);
```

```
// RL: right heavy and right child is left-heavy
if (balance < -1 && getBalanceFactor(ptr->rchild) > 0) {
    ptr->rchild = rightRotate(ptr->rchild);
    return leftRotate(ptr);
}
```

6. Traversals — Recursive and Non-Recursive

6.1 Recursive Traversals

All three follow the same pattern: check for empty tree, check for NULL node, then visit in the correct order.

Inorder (Left -> Root -> Right) — gives sorted ascending output

```
void AVLTree::inorder(node *ptr) {
    if (root == NULL) { cout << "Tree is empty"; return; }
    if (ptr != NULL) {
        inorder(ptr->lchild);    // recurse left
        cout << ptr->info << " "; // print current
        inorder(ptr->rchild);    // recurse right
    }
}
```

Preorder (Root -> Left -> Right) — useful for copying a tree

```
// print first, then recurse left, then right
```

Postorder (Left -> Right -> Root) — useful for deleting a tree

```
// recurse left, recurse right, then print
```

6.2 Non-Recursive Inorder (Feature 2)

```
void AVLTree::inorderNR() {
    stack<node*> stk;
    node *ptr = root;
    while (ptr != NULL || !stk.empty()) {
        while (ptr != NULL) {      // go as far left as possible
            stk.push(ptr);
            ptr = ptr->lchild;
        }
        ptr = stk.top(); stk.pop(); // backtrack
        cout << ptr->info << " ";
        ptr = ptr->rchild;          // move to right subtree
    }
}
```

The stack simulates the call stack that recursion normally uses. Phase 1: push nodes while going left. Phase 2: pop the top (this is the next inorder node), print it, then move to its right child. The outer while loop keeps running until both the stack is empty AND ptr is NULL, meaning every node has been visited.

6.3 Non-Recursive Preorder

```
stk.push(root);
while (!stk.empty()) {
    node *ptr = stk.top(); stk.pop();
    cout << ptr->info << " ";
    if (ptr->rchild) stk.push(ptr->rchild); // push right FIRST
    if (ptr->lchild) stk.push(ptr->lchild); // push left SECOND
}
```

Push root, then enter the loop. Pop a node, print it (preorder: root first), then push right child before left child. Since stack is LIFO (Last In First Out), pushing right first means left is popped first next iteration — correct preorder behavior.

6.4 Non-Recursive Postorder (Two Stacks)

```
// Stack1 drives traversal; Stack2 collects in reverse-postorder
stk1.push(root);
while (!stk1.empty()) {
    node *ptr = stk1.top(); stk1.pop();
    stk2.push(ptr);           // save to stack2
    if (ptr->lchild) stk1.push(ptr->lchild);
    if (ptr->rchild) stk1.push(ptr->rchild);
}
while (!stk2.empty()) {
    cout << stk2.top()->info << " "; stk2.pop();
}
```

Postorder (Left->Right->Root) is the reverse of a modified preorder (Root->Right->Left). So: use Stack1 to traverse in Root->Right->Left order pushing each node onto Stack2. Then drain Stack2, which reverses the order to Left->Right->Root — correct postorder.

7. Features 1–10 Explained

Feature 1 — Duplicate Handling (insertDup / deletionDup)

```
void AVLTree::insertDup(int item) {
    node *location = search(item, root);
    if (location != NULL) {
        location->count++; // key exists – just increment
        return;
    }
    root = insertAVL(root, item); // new key – normal AVL insert
}
```

```
void AVLTree::deletionDup(int item) {
    node *location = search(item, root);
    if (location->count > 1) {
        location->count--; // more copies remain
        return;
    }
    root = deleteAVL(root, item); // last copy – remove node
}
```

Instead of inserting a second node for a duplicate value, we reuse the existing node and just increment its count field. This saves memory and keeps the tree smaller. On deletion, we decrement count first; only when count reaches 1 do we actually remove the node from the tree.

Feature 3 — Reverse AVL Tree

```
// Inside ReverseAVLTree::insert():
if (item > ptr->info)
    ptr = ptr->lchild; // LARGER values go LEFT (reversed rule)
else
    ptr = ptr->rchild; // smaller values go right
```

By flipping the BST ordering rule, a standard inorder traversal (Left->Root->Right) now visits nodes from largest to smallest. No extra sorting is needed — the tree structure itself enforces descending order.

Feature 4 — Student Records

```
void AVLTree::insertStudent(int rollNo, const string &name, float marks) {
    // Creates node with rollNo as key, name & marks as satellite data
    // Then rebalances with insertAVL
}

void AVLTree::searchStudent(int rollNo) {
    node *ptr = search(rollNo, root);
    cout << "Roll No: " << ptr->info << endl;
```

```

    cout << "Name    : " << ptr->name << endl;
    cout << "Marks   : " << ptr->marks << endl;
}

```

The same node struct already has name and marks fields (set to blank/0 in integer mode). In student mode, we use Roll No as the BST key so searching by roll number is $O(\log n)$. Name and marks are satellite data — they ride along with the node but are not used for comparisons.

Feature 5 — Leaf Nodes Display & Delete

```

void AVLTree::displayLeaves(node *ptr) {
    if (ptr->lchild == NULL && ptr->rchild == NULL)
        cout << ptr->info << " "; // it is a leaf — print it
    else {
        displayLeaves(ptr->lchild);
        displayLeaves(ptr->rchild);
    }
}

node* AVLTree::deleteLeavesHelper(node *ptr) {
    if (ptr == NULL) return NULL;
    if (ptr->lchild == NULL && ptr->rchild == NULL) {
        delete ptr; return NULL; // remove leaf
    }
    ptr->lchild = deleteLeavesHelper(ptr->lchild);
    ptr->rchild = deleteLeavesHelper(ptr->rchild);
    return ptr;
}

```

A leaf is a node where both lchild and rchild are NULL. displayLeaves recurses over the tree and prints only those nodes. deleteLeavesHelper recurses post-order style: visit children first, delete any that are leaves, and return the updated pointer to the parent.

Feature 6 — Inorder Successor & Predecessor

The inorder predecessor of X is the largest value smaller than X. The inorder successor is the smallest value larger than X.

```

// Predecessor: rightmost node in left subtree
node *tmp = ptr->lchild;
while (tmp->rchild != NULL) tmp = tmp->rchild;
predecessor = tmp;

```

```

// Successor: leftmost node in right subtree
node *tmp = ptr->rchild;
while (tmp->lchild != NULL) tmp = tmp->lchild;
successor = tmp;

```

While searching for the target node, we also track candidates: every time we go left (item < ptr), the current ptr is a candidate successor (it is larger than item). Every time we go right (item > ptr), the

current ptr is a candidate predecessor. Once the node is found, we refine using the subtree approach above.

Feature 7 — Nodes at Given Level

```
void AVLTree::nodesAtLevel(node *ptr, int target, int current) {
    if (ptr == NULL) return;
    if (current == target) { cout << ptr->info; return; }
    nodesAtLevel(ptr->lchild, target, current + 1);
    nodesAtLevel(ptr->rchild, target, current + 1);
}
```

We carry a current counter that starts at 1 (root level). At each recursive call we pass current + 1 to go deeper. When current matches target, we print the node's value. Otherwise we keep descending. This is essentially a level-filtered preorder traversal.

Feature 8 — Display Left-Child Nodes

```
void AVLTree::displayLeftNodes(node *ptr) {
    if (ptr == NULL) return;
    if (ptr->lchild != NULL)
        cout << ptr->lchild->info << " "; // print LEFT child
    displayLeftNodes(ptr->lchild); // recurse into left subtree
    displayLeftNodes(ptr->rchild); // recurse into right subtree
}
```

Visits every node in the tree. For each node that HAS a left child, it prints that left child's value. By visiting both subtrees, we cover the entire tree.

Feature 9 — Largest Value at Given Level

```
int AVLTree::largestAtLevel(node *ptr, int target, int current) {
    if (ptr == NULL) return INT_MIN; // sentinel: no node here
    if (current == target) return ptr->info;
    int leftMax = largestAtLevel(ptr->lchild, target, current+1);
    int rightMax = largestAtLevel(ptr->rchild, target, current+1);
    return (leftMax > rightMax) ? leftMax : rightMax;
}
```

Returns INT_MIN (the smallest possible integer, from <climits>) when there is no node at the required level. This lets the caller safely compare return values with > and detect 'level does not exist' by checking for INT_MIN. Both left and right results are compared and the larger is returned.

Feature 10 — Display Right-Child Nodes

```
void AVLTree::displayRightNodes(node *ptr) {
    if (ptr == NULL) return;
    if (ptr->rchild != NULL)
```

```
        cout << ptr->rchild->info << " "; // print RIGHT child
        displayRightNodes(ptr->rchild);
        displayRightNodes(ptr->lchild);
    }
```

Mirror of Feature 8 but for right children. Note the recursion visits rchild before lchild, which gives a right-biased traversal, but since we are just collecting right-child values the order does not affect correctness.

8. main.cpp — Menu-Driven Interface

8.1 Setup

```
AVLTree avlTree;  
ReverseAVLTree revTree;
```

Two tree objects are created. `avlTree` is the main integer/student AVL tree. `revTree` is a separate reverse-order BST used only by menu option 8.

8.2 Infinite Loop with switch-case

```
while (true) {  
    // print menu  
    cin >> choice;  
    switch (choice) {  
        case 1: /* insert */ ...  
        case 6: return 0; // only exit  
        ...  
    }  
}
```

The program runs indefinitely until the user enters 6 (Quit), which calls `return 0` from `main()`. Every other case reads input, calls the appropriate `AVLTree` method, and loops back to the menu. Invalid choices fall through to the default case which prints an error message.

8.3 Sub-menus

Options 7, 8, 9, 10 have their own sub-menus with a second `cin >> subChoice` inside a nested switch or if-else. For example, option 9 (Student Records) asks whether to insert or search before calling `insertStudent` or `searchStudent`.

The student name input uses `cin.ignore()` followed by `getline(cin, sName)`. `cin.ignore()` is needed because after reading the integer `rollNo` with `cin >> rollNo`, a newline character remains in the input buffer. Without `cin.ignore()`, `getline` would immediately read that newline as an empty string.

9. Time and Space Complexity

Operation	Time Complexity	Reason
Search	$O(\log n)$	AVL height is always $\leq 1.44 \log_2(n)$
Insert	$O(\log n)$	Descend + at most 2 rotations going back up
Delete	$O(\log n)$	Descend + rotations on the way back up
Traversal	$O(n)$	Must visit every node exactly once
Nodes at level k	$O(n)$	Worst case visits all nodes
Space (tree)	$O(n)$	One node struct per stored value
Space (NR traversal)	$O(h) = O(\log n)$	Stack depth = tree height

10. Viva Preparation — Q&A

Study these questions and answers. They cover everything your examiner is likely to ask.

Q: What is an AVL tree and how does it differ from a BST?

A: An AVL tree is a self-balancing BST where the balance factor of every node is -1, 0, or +1. A regular BST can become a linear list ($O(n)$ operations) in the worst case. AVL trees always maintain $O(\log n)$ height by performing rotations after inserts and deletes.

Q: What does `getBalanceFactor()` return and what do the values mean?

A: It returns `left_height - right_height`. +2 means left subtree is 2 levels taller (left-heavy imbalance). -2 means right subtree is 2 levels taller (right-heavy imbalance). 0 means perfectly balanced. +1 or -1 is acceptable.

Q: Why does `insertAVL` return a node pointer instead of being void?

A: Each recursive call may perform a rotation that changes which node is the root of that subtree. By returning the new subtree root, the caller (parent node) can update its child pointer. If it were void, the parent would still point to the old, now-displaced node.

Q: Explain Left-Right (LR) rotation in plain words.

A: LR happens when the left child is right-heavy. You cannot fix it with one rotation because the imbalance is 'zigzag-shaped'. First you `leftRotate` the left child to make it straight (converting to LL case), then you `rightRotate` the original node to complete the fix.

Q: Why do we use `findMin` of the right subtree when deleting a two-child node?

A: We need a replacement that maintains BST ordering. The inorder successor (minimum of right subtree) is the smallest value that is still larger than the deleted node. Replacing the node with it preserves the BST property without structural changes beyond deleting the successor.

Q: Why does non-recursive postorder need two stacks?

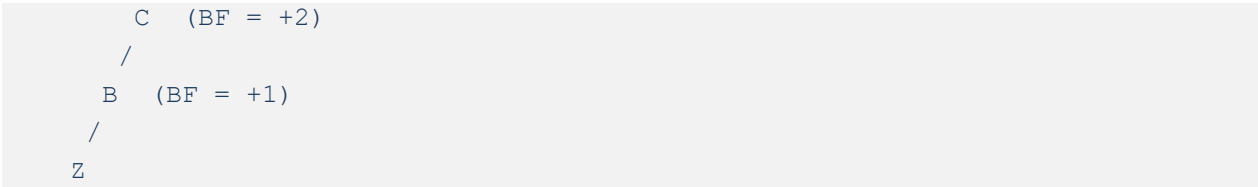
A: Postorder (L->R->Root) is the reverse of a modified preorder (Root->R->L). Stack1 generates Root->Right->Left order. Each node is pushed onto Stack2. When Stack1 is done, draining Stack2 reverses the order to Left->Right->Root, which is correct postorder.

Q: How does the duplicate-handling count field work?	A: Every node starts with count=1. insertDup checks if the key exists first; if yes, it just does count++. deletionDup decrements count; only when count becomes 1 and we call deletionDup again does the node actually get removed from the tree by deleteAVL.
Q: How does the Reverse AVL Tree print values in descending order?	A: The BST insertion rule is inverted: larger values go to the LEFT child. A standard inorder traversal (Left->Root->Right) therefore visits the largest node first, the second-largest second, and so on — giving descending output automatically.
Q: Why does the student name input use cin.ignore() before getline?	A: cin >> rollNo reads the integer but leaves a newline in the buffer. getline() reads until newline, so it would immediately see that leftover newline and return an empty string. cin.ignore() discards that newline, allowing getline to read the actual name.
Q: What does INT_MIN represent in largestAtLevel and why use it?	A: INT_MIN is the smallest possible integer value (-2147483648 on 32-bit systems), defined in <climits>. Using it as a sentinel return for NULL subtrees means any real node value will be larger than INT_MIN, so the max comparison always correctly picks a real node over a missing one.
Q: What are case_a, case_b, case_c and why are they empty?	A: These were deletion helper methods from an earlier plain-BST implementation (before AVL balancing was added). They handled leaf deletion, single-child deletion, and two-child deletion manually. In the AVL version, all deletion is handled by deleteAVL using rotations. They are kept as stubs for backward compatibility with the header file.
Q: What is the worst-case height of this AVL tree for n nodes?	A: The maximum height of an AVL tree with n nodes is approximately $1.44 * \log_2(n+2)$. This guarantees that even in the worst case, no search, insert, or delete operation takes more than $O(\log n)$ comparisons.

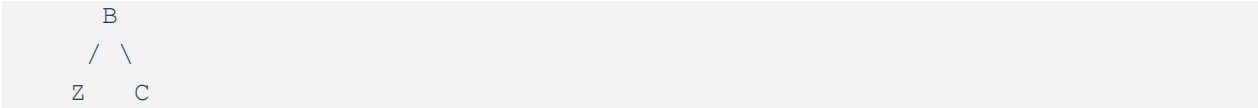
11. Rotation Diagrams

Left-Left (LL) Case — Single Right Rotation

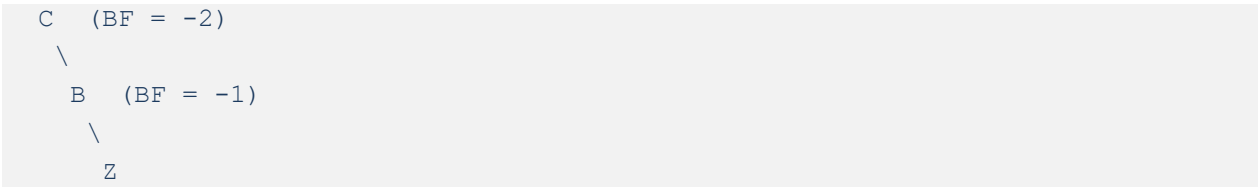
Before rotation (Z is newly inserted, tree is left-heavy at C):



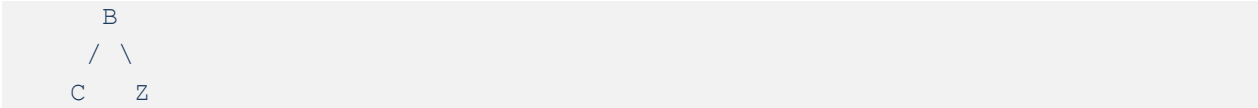
After rightRotate(C):



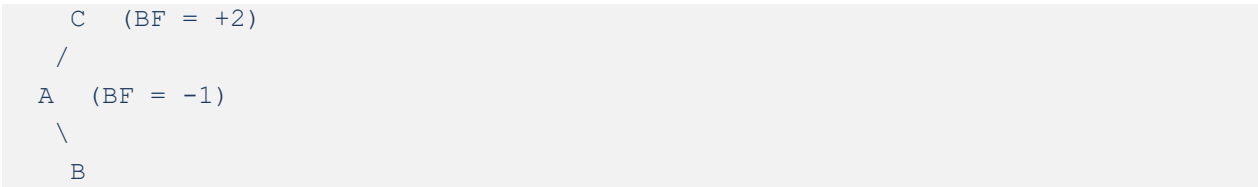
Right-Right (RR) Case — Single Left Rotation



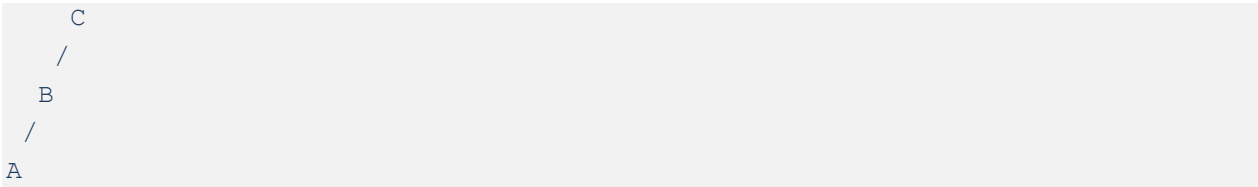
After leftRotate(C):



Left-Right (LR) Case — Double Rotation



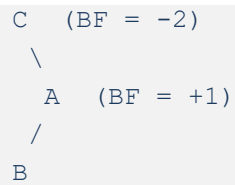
Step 1: leftRotate(A):



Step 2: rightRotate(C) — now LL case:



Right-Left (RL) Case — Double Rotation



Step 1: rightRotate(A). Step 2: leftRotate(C). Result: B becomes root with C on left and A on right.

Good luck in your viva!

You built a complete self-balancing AVL Tree with 10 features. Be confident.